

Trabalho Final - Predição de Popularidade de Músicas Estilo Spotify

Integrantes

Thiago Poliseli Silva - RA: PREENCHER

Integrante 2 - RA: PREENCHER

Integrante 3 - RA: PREENCHER

Integrante 4 - RA: PREENCHER

Resumo do Projeto

O projeto aplica Inteligência Artificial para prever se uma música será popular ou não popular com base em características musicais inspiradas em datasets do Spotify. Foram utilizados atributos como danceability, energy, loudness, acousticness, valence, tempo, duration_ms e track_genre.

O problema foi tratado como classificação binária. A variável alvo popularidade_alta recebe valor 1 quando popularity ≥ 70 e valor 0 quando popularity < 70 .

Contextualização, Problema e Hipótese

Plataformas de streaming armazenam grandes volumes de dados sobre músicas. A hipótese da equipe é que músicas com maior energia, dançabilidade, valência e intensidade sonora tendem a ter maior popularidade. Assim, modelos de IA podem identificar padrões nos atributos musicais e classificar músicas como populares ou não populares.

Dataset

O dataset foi criado pela equipe por meio do script src/generate_dataset.py, com 5.000 registros e 15 colunas. A base é sintética, mas segue a estrutura de datasets musicais estilo Spotify. O enunciado permite a criação de dataset próprio, o que torna a execução reprodutível e sem dependência de download externo.

Principais atributos: track_genre, popularity, danceability, energy, loudness, speechiness, acousticness, instrumentalness, liveness, valence, tempo e duration_ms.

Preparação dos Dados

As etapas realizadas foram: carregamento do CSV, validação das colunas obrigatórias, remoção de registros ausentes, criação da variável alvo, padronização dos atributos numéricos com StandardScaler, codificação do gênero musical com OneHotEncoder e divisão estratificada entre treino e teste.

Divisão: 75% para treino, totalizando 3.750 amostras, e 25% para teste, totalizando 1.250 amostras.

Métodos de IA Utilizados

Método da Parte 1: Árvore de Decisão, usando `DecisionTreeClassifier(max_depth=6, random_state=42)`. O modelo é interpretável e permite visualizar regras de decisão.

Método da Parte 2: Rede Neural MLP, usando `MLPClassifier` com camadas ocultas (32, 16), ativação ReLU, otimizador Adam, early stopping e `random_state=42`. O modelo consegue aprender relações não lineares entre os atributos.

Avaliação dos Modelos

As métricas utilizadas foram acurácia, precisão, recall, F1-score e matriz de confusão. Também foram gerados gráficos de distribuição da base e comparação dos modelos.

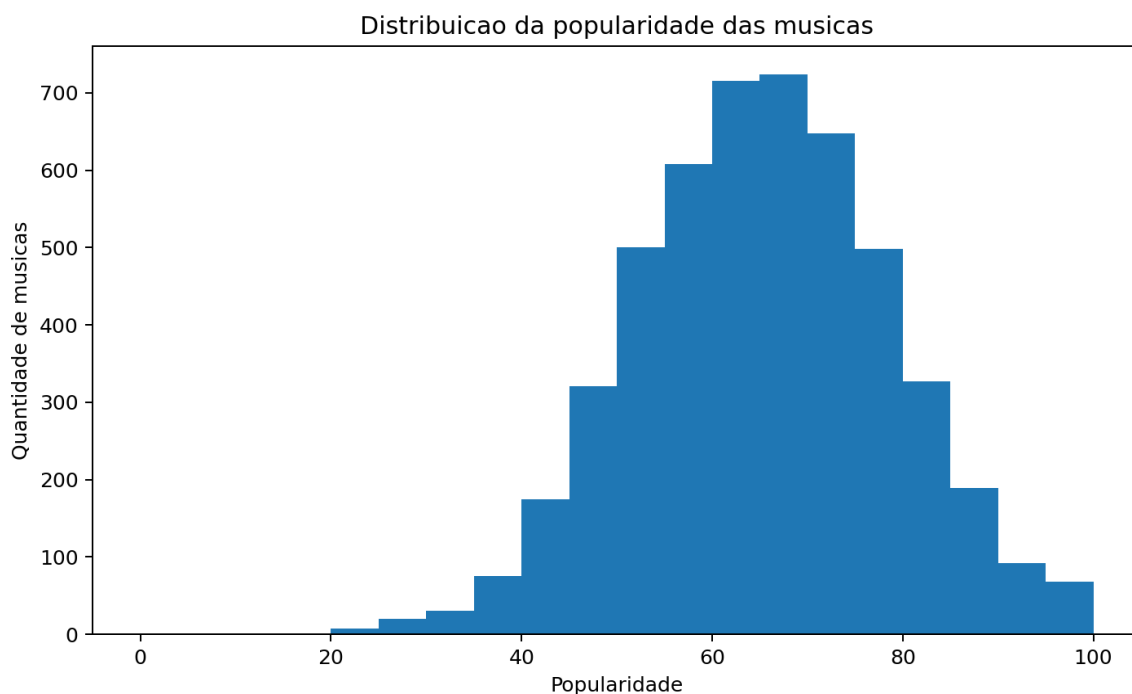


Figura 1 - Distribuição da popularidade das músicas.

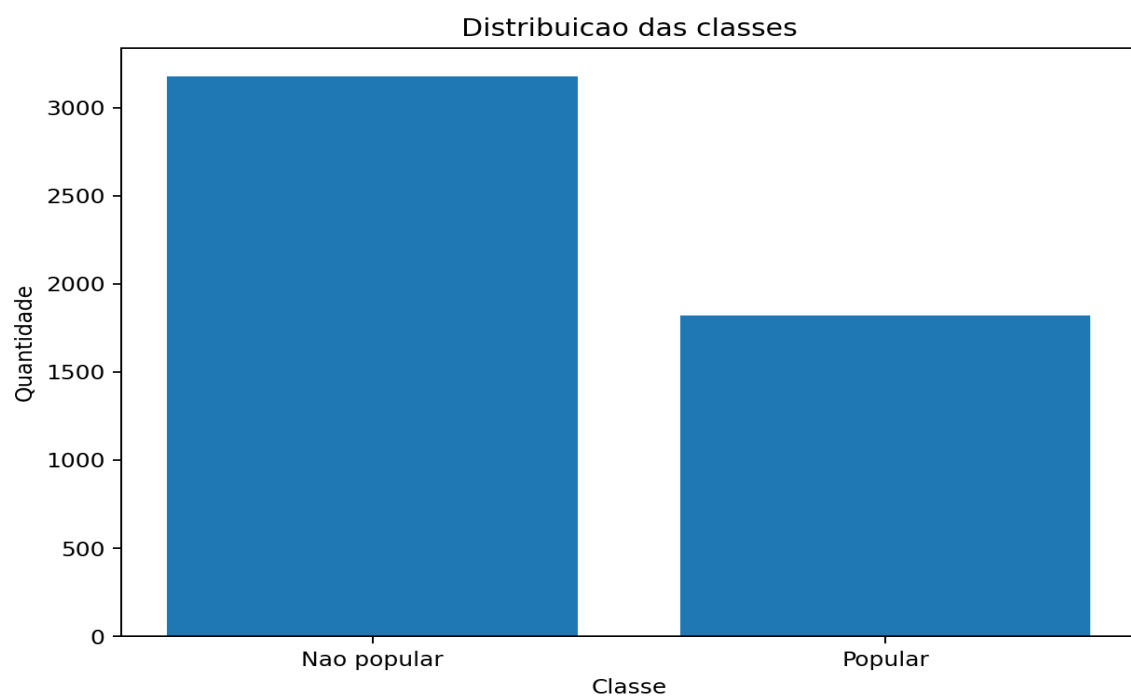


Figura 2 - Distribuição das classes popular e não popular.

Matrizes de Confusão

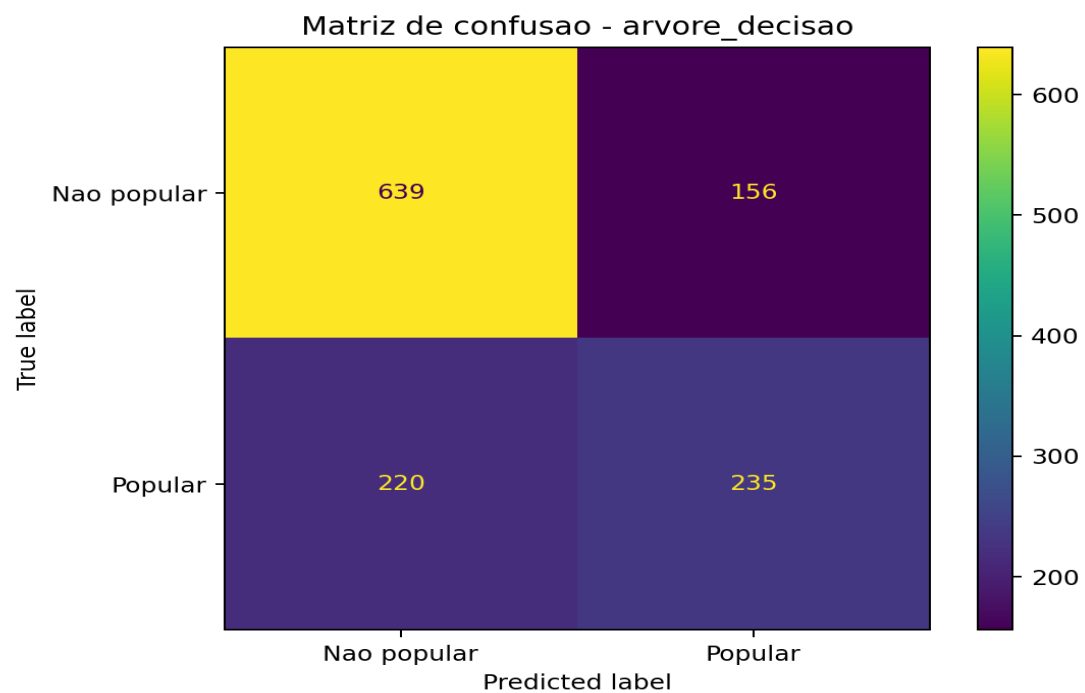


Figura 3 - Matriz de confusão da Árvore de Decisão.

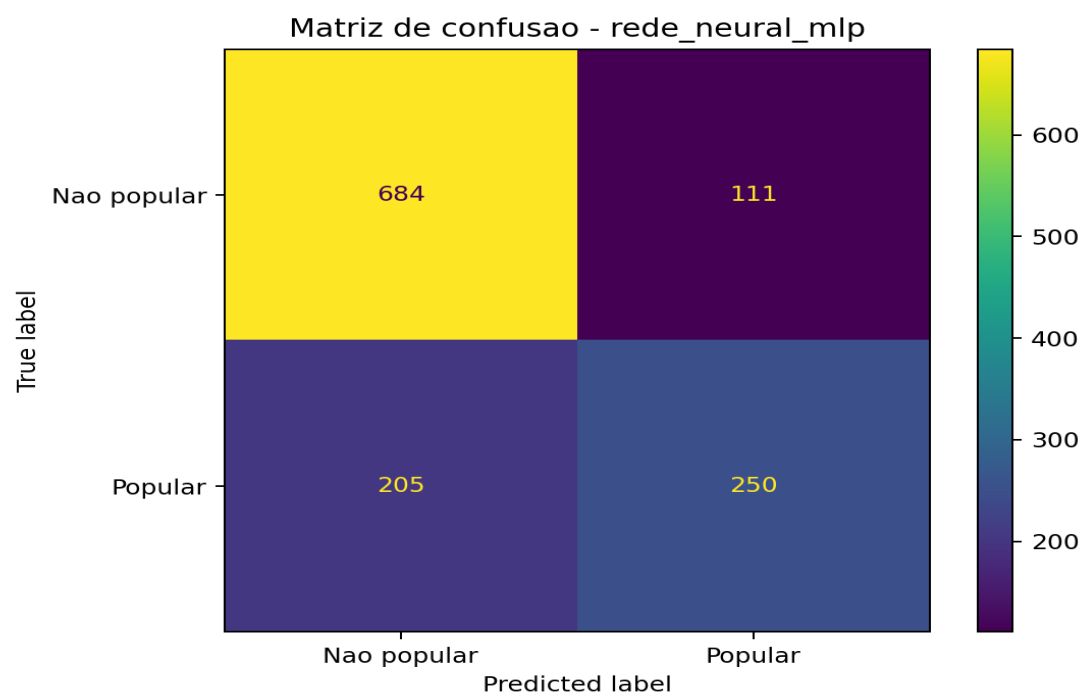


Figura 4 - Matriz de confusão da Rede Neural MLP.

Tabela de Métricas

Modelo	Acurácia	Precisão	Recall	F1-score
Árvore de Decisão	0.6992	0.6010	0.5165	0.5556

Rede Neural MLP	0.7472	0.6925	0.5495	0.6127
-----------------	--------	--------	--------	--------

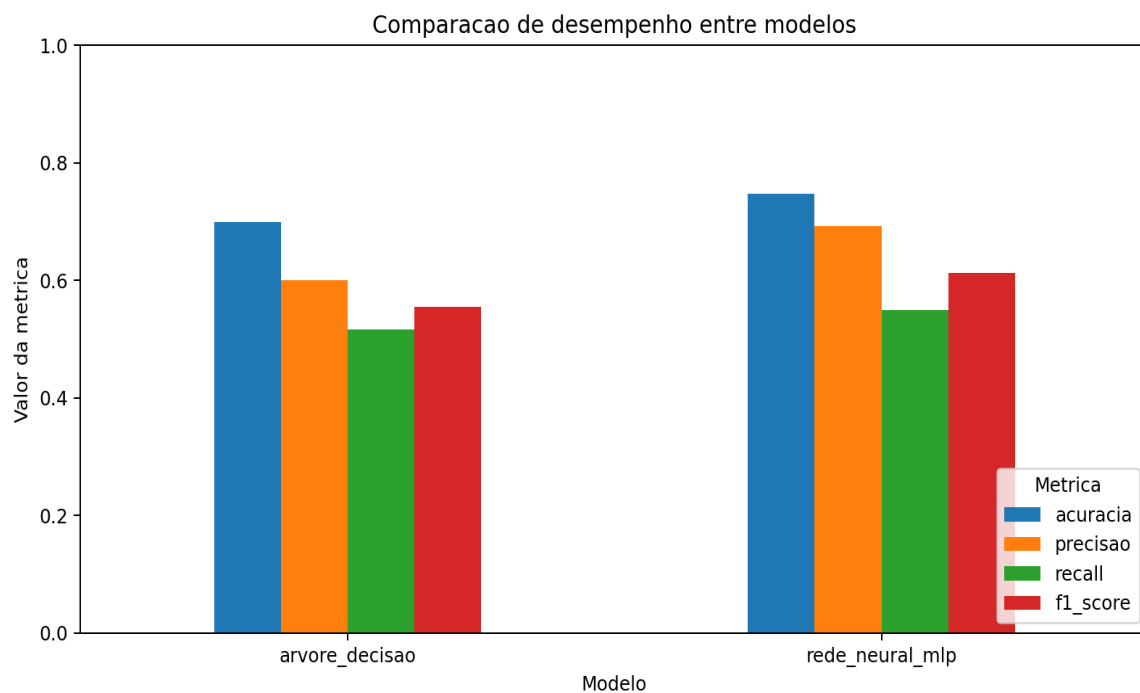


Figura 5 - Comparação gráfica entre os modelos.

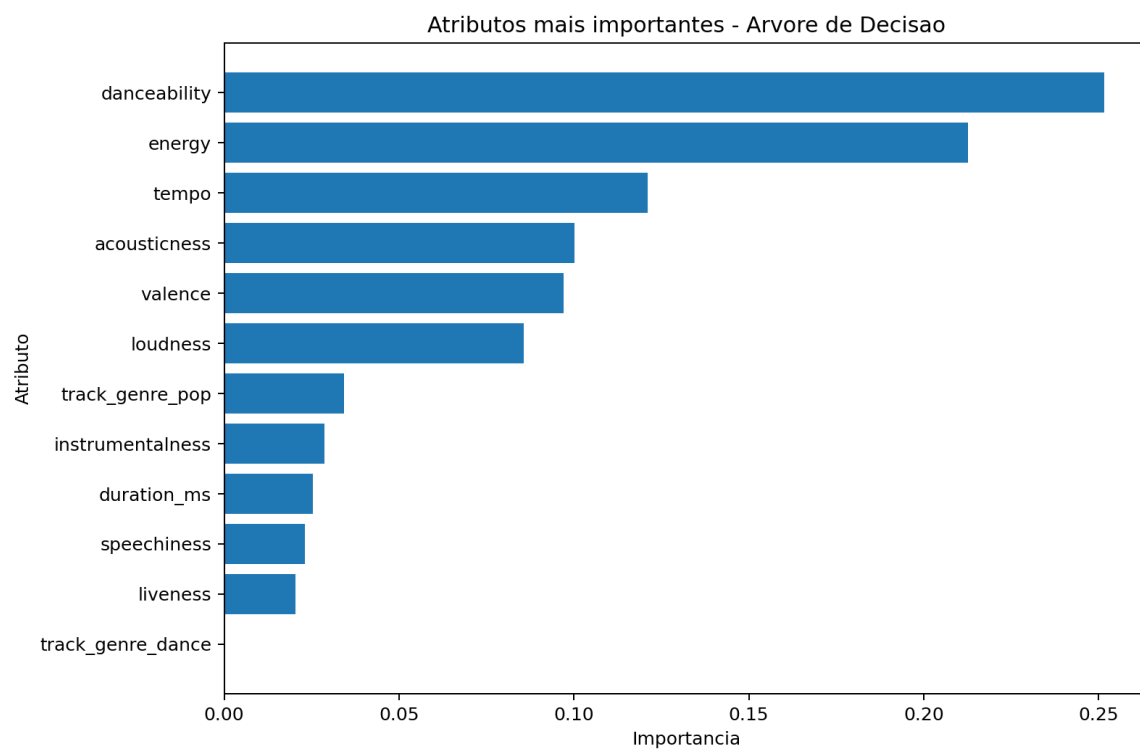


Figura 6 - Atributos mais importantes na Árvore de Decisão.

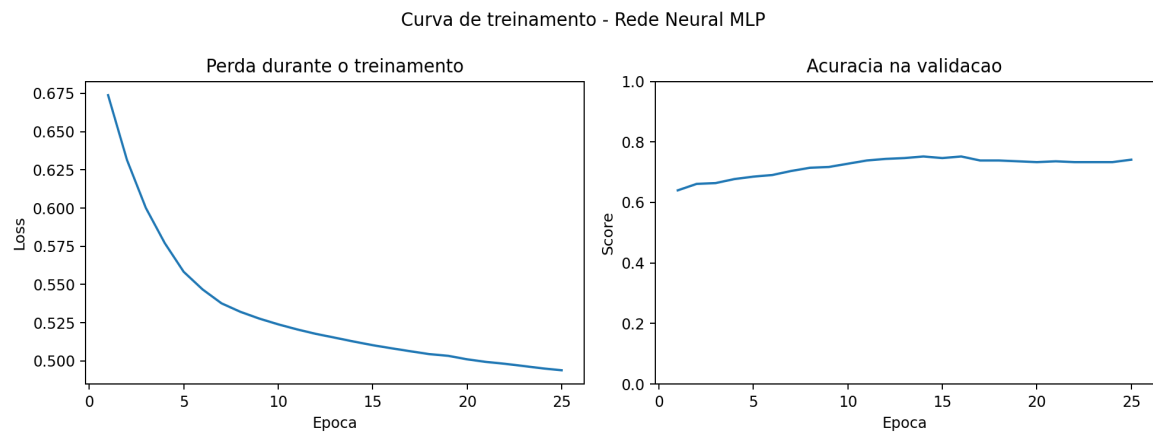


Figura 7 - Curva de treinamento da Rede Neural MLP.

Comparação dos Resultados

A Rede Neural MLP apresentou melhor desempenho geral em comparação com a Árvore de Decisão. O modelo MLP obteve maior acurácia, precisão e F1-score. A Árvore de Decisão teve desempenho inferior, mas possui a vantagem de ser mais interpretável.

Com base no F1-score, que equilibra precisão e recall, o melhor modelo foi a Rede Neural MLP.

Modelo Treinado

Os modelos treinados foram salvos na pasta `models/`: `arvore_decisao.pkl` e `rede_neural_mlp.pkl`. Eles podem ser carregados com a biblioteca `joblib`.

Material de Estudo e Apresentação

A pasta `docs/` contém uma checklist do enunciado, um guia de estudo dos algoritmos e um roteiro sugerido para a apresentação.

Arquivos: `docs/checklist_enunciado.md`, `docs/guia_estudo_algoritmos.md` e `docs/roteiro_apresentacao.md`.

Como Executar

1. Criar ambiente virtual: `python -m venv .venv`
2. Instalar dependências: `pip install -r requirements.txt`
3. Gerar dataset: `python src/generate_dataset.py`
4. Treinar e avaliar: `python src/main.py`
5. Testar predição: `python src/predict.py`
6. Gerar PDF do relatório: `python src/generate_pdf.py`

Conclusão

O projeto demonstrou o processo completo de criação de uma solução baseada em IA: definição do problema, criação e preparação dos dados, treinamento dos modelos, avaliação com métricas e gráficos, comparação e conclusão.

A Rede Neural MLP foi o modelo mais adequado para este dataset, alcançando F1-score de 0.6127 contra 0.5556 da Árvore de Decisão. Apesar disso, a Árvore de Decisão é útil para fins de interpretação e explicação acadêmica.